

J.A.R.V.I.S.

AI OPERATING SYSTEM // MARK XLVII

Comprehensive Technical Specification & Blueprint: Multimodal Conversational AI, Bilingual Speech Synapse, Arc Reactor Synapse Calibration & 150% Overdrive, Dynamic Quick App Customization with Live Favicons, Windows Desktop & Browser Automation, Real-Time Hardware Telemetry, Biometrics & Tactical Glassmorphic Web HUD.

SYSTEM VERSION v3.2.5 PRODUCTION STABLE	DEPLOYMENT PORTS Backend :5000 | Web HUD :8080
COGNITION PIPELINE Gemini 2.5/2.0 Flash + Local Knowledge	STORAGE & AUTH MongoDB Atlas + OpenCV Biometrics

DOCUMENT REF: SPEC-JARVIS-3.2.5-MASTER

DATE: OCTOBER 2026 // LEAD ARCHITECT: ROHIT KUMAR GUPTA

1. Executive Summary & All 21 Core Capability Pillars

J.A.R.V.I.S. (Just A Rather Very Intelligent System) is an automated full-stack AI OS inspired by the Stark Industries Mark XLVII interface. It bridges multimodal AI cognition, speech synthesis, real-time computer vision, OS automation, and cloud data stores into an integrated tactical HUD.

01 AI Assistant
Context-aware conversational intelligence with persistent memory and Stark persona.

02 Voice Input (STT)
Web Speech & offline Whisper/Vosk recognition in English (en-IN/en-US) & Hindi (hi-IN).

03 Text-to-Speech (TTS)
Multi-engine TTS (WebSpeech, EdgeTTS) with automated Devanagari script routing.

04 LLM / API Cognition
Google Gemini 2.5/2.0 Flash failover with instant offline local knowledge fallback.

05 Desktop Automation
Window controls (min/max/tile), system volume, workstation lock, sleep, reboot, shutdown.

06 File & App Manager
Launches VS Code, Chrome, Spotify, Calculator, plus safe file search & dataset disk upload.

07 Browser Automation
Playwright engine for web navigation, URL normalization, multi-tabs, and DOM parsing.

08 YouTube Controls
Direct video search, automatic playback, volume/fullscreen, and transcript extraction.

09 WhatsApp Messages
Automated messaging via WhatsApp Web & Desktop with contact search and failsafes.

10 Screen Vision & OCR
High-speed screen capture (MSS), window inspection, and Gemini Vision reasoning.

11 Weather Telemetry
Live atmospheric telemetry via Open-Meteo REST service with top header scan & chat cards.

12 Tool Sandbox Execution
Dynamic tool dispatcher, isolated Python execution sandbox, and algorithm builder.

13 Modular Architecture
Clean decoupled structure across frontend, backend, core, actions, and memory layers.

14 Arc Reactor Synapse
Multi-ring core with 150% Overdrive mode, synapse calibration sequence & audio ripples.

15 6 Assistant States
Real-time transitions: Standby, Listening, Thinking, Tool Execution, Responding, Success.

16 Tactical HUD & Quick Apps
Glassmorphic HUD with dynamic Quick App customization, Google S2 favicons & side matrix.

17 Biometrics & Auth
OpenCV Haar Cascade face scan, Master PIN (2710) manual bypass, and 2FA OTP auth terminal.

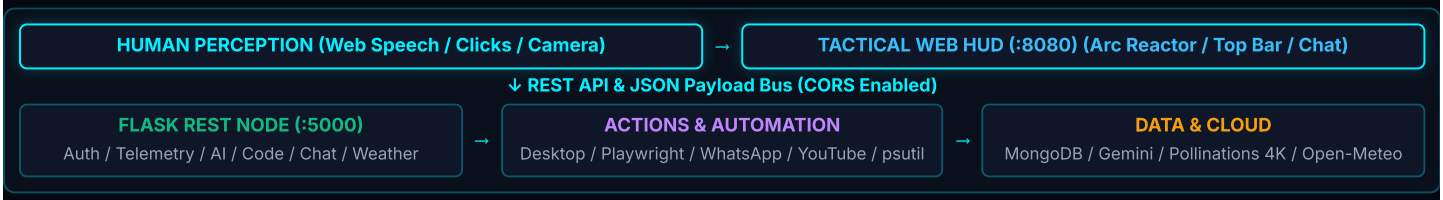
18 Conversation Logs
Persistent session logs via LocalStorage + SQLite/JSON hybrid store with PDF export.

19 MongoDB Hybrid Data
Cloud MongoDB Atlas integration with automated local fallback to backend JSON stores.

20 Flask REST APIs (:5000)
CORS-enabled REST endpoints for system commands, AI, code, auth, chat, and weather.

21 360 Tactical Themes
5 preset tactical themes (Cyan, Green, Red, Gold, Purple) + dynamic 360 color wheel.

2. High-Level Architecture Runtime Pipeline



3. Arc Reactor HUD & Dynamic Assistant States

The central synapse visualizer reacts dynamically across **6 distinct operational states**:

STATE 01 Standby (Idle)
- **Visual Core:** Rhythmic, slow pulsing cyan glow at 0.5Hz.
- **System Meaning:** Workstation is armed, listening daemon ready.
- **Trigger:** Startup completion or post-response rest.

STATE 02 Listening
- **Visual Core:** Active red/cyan audio wave frequency oscillation.
- **System Meaning:** Web Speech API active, receiving en-IN/hi-IN audio.
- **Trigger:** Arc Reactor click or floating microphone dock activation.

STATE 03 Thinking / Synapsing
- **Visual Core:** Rapid concentric ring matrix rotation in violet/magenta.
- **System Meaning:** Google Gemini reasoning or local knowledge synthesis.
- **Trigger:** Query submission (voice or terminal text bar).

STATE 04 Tool Execution
- **Visual Core:** High-velocity orbital pulse with amber warning accents.
- **System Meaning:** Running Playwright, WhatsApp send, or Python sandbox.
- **Trigger:** Dispatching action via actions/ executor.

STATE 05 Responding / Vocalizing
- **Visual Core:** Kinetic emerald ripple waves synchronized with speech audio.
- **System Meaning:** SpeechSynthesisUtterance active; chat bubble streaming.
- **Trigger:** TTS playback of assistant answer.

STATE 06 Success / Confirmation
- **Visual Core:** Instantaneous bright emerald flash and chime.
- **System Meaning:** Target action completed successfully.
- **Trigger:** App opened, volume set, message dispatched, screenshot taken.

4. Web-Based J.A.R.V.I.S. Tactical HUD (Port 8080)

HUD Glassmorphic HUD & Arc Reactor
Backdrop-filter blur, particle canvas field, digital HUD clock, battery metrics, Arc Reactor zoom modal with 150% Overdrive (4.82 GW) & calibration.

APPS Quick Applications Customization
Dynamic user site shortcut manager with live Google S2 Favicon integration, 1-click popular presets (ChatGPT/Claude/Discord) & persistent storage.

CONTROL Tactical Side Drawer Matrix
Slide-out Stark navigation drawer with operator profile, voice mute/active toggle, biometric portal link, and system diagnostics.

THEMES 360 Theme Engine
5 tactical presets (Cyan Matrix, Green Diagnostics, Red Combat, Gold Centurion, Purple Quantum) + custom continuous 360 color wheel.

COMMS Tactical Hologram Communications Link (Mark XLVII Direct Satellite)
Widescreen crystal clear video stage (540px x 310px) with top-right live duration overlay (🕒 00:00), Stark Tactical HUD Identity Banner (◈ CONTACT NAME ◈) with dual cyan pillars, and interactive Mic Mute, Camera Suspend & End Call controls.

5. Authentication, Biometrics & MongoDB Integration

SECURITY VECTOR	IMPLEMENTATION	ROLE & SPECIFICATION
Optical Facial Scan	OpenCV Haar Cascades (`POST /api/auth/face`)	Real-time in-memory webcam frame facial detection and biometric user verification.
Passcode Fallback	PIN Entry Mode	4-digit Master PIN security entry (`2710`) with manual emergency bypass. Requires explicit BYPASS button click or Enter key.
2FA OTP Verification	Hybrid Auth Flow (`POST /api/auth/verify-otp`)	6-digit numeric OTP generation & verification for new accounts and credential login.
Tactical Auth Portal	`frontend/auth.html`	Standalone WebGL GLSL shader portal with interactive pull-cord theme toggle and 5 swatches.
MongoDB Hybrid Data	MongoDB Atlas + JSON/SQLite Fallback	Cloud user profile persistence with local auto-failover to `backend/data/users.json` .
Conversation History	`/api/chat/history` + LocalStorage	Full chat log persistence with 1-click PDF export, log search, and history clearance.

6. Native Tool & Automation Subsystems (`actions/`)

01 Browser Automation (Playwright)

File: `actions/browser_control.py`

- Automated Chrome/Edge headless and headful session management.
- Intelligent URL normalization (e.g. "instagram" to "https://instagram.com").
- DOM interaction: clicking, form filling, scroll management, link scraping.
- Fast fallback to OS default browser via Python webbrowser.

02 YouTube Automation & Transcripts

File: `actions/youtube.py`

- Direct video playback matching user query terms.
- Video transcript extractor via youtube_transcript_api.
- Controls: play, pause, seek, volume adjustment, fullscreen toggling.

03 WhatsApp & Message Dispatch

File: `actions/messaging.py`

- Automates WhatsApp Web & desktop client via PyAutoGUI.
- Contact resolution: search contact, open conversation, type and paste message.
- Encrypted email dispatch via SMTP integration.
- PyAutoGUI failsafe protections and automated clipboard management.

04 Screen Processor & Vision

File: `actions/screen_processor.py`

- Ultra-fast screen capture using mss and PIL.
- Multimodal vision queries routed to Google Gemini Vision.
- Webcam snapshot capture for facial presence verification.

05 Windows Control & Quick App Customizer

Files: `actions/open_app.py`, `desktop.py`, `utils.js`

- App launcher: dynamic custom site additions, Google S2 Favicon auto-sync, 1-click presets.
- Voice routing: "add site" triggers configuration terminal; "open <site>" launches targets.
- Window management: minimize, maximize, restore, close active window.
- System volume control (mute, set percentage), screenshot capture, workstation lock.

06 Code Interpreter & Sandbox

File: `actions/code_helper.py`

- Automated algorithm generator & LeetCode/Codeforces solver.
- Secure Python subshell execution with 10-second timeout guard.
- Prism.js syntax highlighting with 1-click Copy and Fullscreen Zoom.

7. REST API Endpoints Specification (Port 5000)

CATEGORY	METHOD	ENDPOINT	DESCRIPTION
System	GET	`/`	Health check & version status report
System	POST	`/api/system/command`	Hardware diagnostics (CPU/RAM/Temp/Speed) and OS actions
System	POST	`/api/system/shell`	Execute sandboxed shell commands with 10s timeout
AI	POST	`/api/ai/generate`	Server-side Google Gemini multi-model routing
Code	POST	`/api/code/helper`	Code generation, explanation, and sandbox execution
Auth	POST	`/api/auth/face`	OpenCV Haar Cascade facial biometric verification
Auth	POST	`/api/auth/verify-otp`	Verify 6-digit OTP and complete authorization
Chat	GET / POST / DEL	`/api/chat/history`	Retrieve, append, or clear conversation logs
Weather	GET	`/api/weather`	Live Open-Meteo temperature, atmospheric conditions, and humidity
Calls	POST	`/api/calls/call`	Initiate direct satellite voice/video call session with target identity
Auth	POST	`/api/auth/update-avatar`	Persist cropped 256×256 user identity photo to database

8. Modular Repository Architecture

```
Jarvis/
├── 🚀 ORCHESTRATION & PACKAGING
│   ├── run_dashboard.py       # Full-Stack Launcher (5000+8080)
│   ├── main.py                # PyQt6 Live Multimodal Client
│   ├── ui.py                  # Sci-Fi PyQt6 Desktop Client GUI
│   ├── setup.py               # Setuptools Packaging Spec
│   ├── package.json           # Project Metadata & NPM Scripts
│   ├── requirements.txt       # Python Dependencies Manifest
│   ├── requirements-docker.txt # Minimal Docker Dependencies
│   ├── Dockerfile             # Container Deployment Blueprint
│   ├── docker-compose.yml     # Multi-Container Compose
│   ├── .env / .env.example    # Local Secret / Template Vars
│   └── .gitignore             # Git Ignore Rules
├── 📚 DOCUMENTATION & BLUEPRINTS
│   ├── README.md              # GitHub Presentation & Quickstart
│   ├── AGENTS.md              # AI Assistant Architecture & Invariants
│   ├── PROJECT_OVERVIEW.md    # Subsystem Architecture Mapping
│   ├── JARVIS_TECHNICAL_PROJECT_REPORT.md # Technical Report
│   ├── LICENSE / CHANGELOG.md / CONTRIBUTING.md
│   └── docs/                  # Technical Subsystem Manuals
│       ├── API_DOCUMENTATION.md / ARCHITECTURE.md
│       ├── DESIGN.md / DEVELOPER_GUIDE.md / INSTALLATION.md
│       └── PYTHON_TUTORIAL.md / USER_GUIDE.md
├── 🖥️ FRONTEND (PORT 8080)
│   ├── index.html / auth.html / favicon.ico
│   ├── css/ (style, dashboard, chat, lockscreen, settings...)
│   ├── js/ (app, ai, voice, commands, dashboard, chat...)
│   └── assets/ (images, icons, sounds)
└── ⚙️ BACKEND (PORT 5000)
    ├── server.py / server_config.py
    ├── api/ (ai, auth, calls, chat, code, files, system, weather)
    ├── automation/ (os_controls.py, system_monitoring.py)
    ├── database/ (db_manager.py) & data/ (users.json)
```

```
├── 🧠 CORE (MULTIMODAL LIVE ENGINES)
│   ├── __init__.py            # Engine Package Exports
│   ├── llm.py                 # Multi-Model LLM Orchestration
│   ├── stt.py                 # Offline Whisper & Vosk STT
│   ├── tts.py                 # EdgeTTS, Kokoro & ElevenLabs Voices
│   ├── prompt.txt             # Master System Persona Prompt
│   └── installer.py / startup.py
├── ⚡ ACTIONS (NATIVE AGENT TOOL SUITE)
│   ├── browser_control.py     # Playwright Web Automation
│   ├── code_helper.py         # Python Sandbox & CP Helper
│   ├── computer_control.py    # Process & System Power
│   ├── computer_settings.py   # Windows Audio & Display
│   ├── desktop.py / dev_agent.py / file_controller.py
│   ├── file_processor.py      # Document Parsing & Summarization
│   ├── flight_finder.py / game_updater.py / messaging.py
│   ├── music.py / open_app.py / proactive.py / reminder.py
│   ├── screen_processor.py    # Screen Capture OCR & Vision
│   ├── system_monitor.py      # Real-Time CPU/RAM Telemetry
│   └── weather.py / web_search.py / youtube.py
├── 💾 MEMORY & CONFIG
│   ├── memory/ (memory_manager, config_manager, user_profile)
│   └── config/ (api_keys.json.example, environment, jarvis.ico)
└── 🧪 TESTS & VERIFICATION
    ├── tests/ (test_actions, test_ai, test_api, test_auth, test_backend, test_voice)
    ├── screenshots/           # High-Resolution HUD Captures
    └── demo/                  # Video Previews & Animation Assets
```

9. Installation, Quickstart & Launch Guide

STEP 1 **Prerequisites**

Windows 10/11, Python 3.12+ (or 3.10+), modern browser (Chrome, Edge, Brave).

STEP 2 **Dependencies**

```
pip install -r requirements.txt
```

STEP 3 **Single-Command Full-Stack Launch**

Starts the Flask backend on **Port 5000** and the Tactical Web HUD on **Port 8080** simultaneously:

```
py -3.12 run_dashboard.py
# Or standard launcher:
python run_dashboard.py
```

STEP 4 **Access Operating System in Browser**

Open **`http://127.0.0.1:8080`**. Enter your Gemini API key in Settings or immediately use the built-in offline fallback engine.

STARK INDUSTRIES AI DIVISION // ALL RIGHTS RESERVED